

SentinelX

5. AI Tools Used

AI Models and Frameworks

- A Bidirectional Long Short-Term Memory (Bi-LSTM) architecture for on-device threat / non-threat language classification, trained with TensorFlow and Keras, then converted to a lightweight on-device format and bundled directly into the mobile app.
- Google Gemini API (free tier), used server-side to analyze uploaded video, audio, and image evidence for Safe Chat reports.

The Bi-LSTM model was specifically selected because contextual threat narratives depend heavily on sequential language understanding rather than isolated keyword matching, and because on-device inference was required to keep raw audio off any server entirely. Gemini was selected for evidence analysis because it can reason over unstructured media — video, audio, and images — and return plain-language, analyst-readable context.

Backend and Infrastructure

The backend infrastructure was developed using Python, Flask, MongoDB, and Railway for cloud hosting and deployment. The Flask backend handles Safe Chat report ingestion, evidence storage, organizational dashboard communication, routing uploaded evidence to the Gemini API only after analyst authorization, and receiving Detect Threat flags from the mobile app. It does not run the Bi-LSTM model — that inference happens entirely on-device. MongoDB supports flexible structured report storage, contextual metadata handling, triage workflow states, and dynamic organizational case management.

Frontend and Mobile Development

The mobile application — including the on-device Bi-LSTM inference for Detect Threat — was developed using React Native (Expo). The NGO organizational dashboard was designed to support analyst review workflows, triaged case visualization, evidence analysis rendering, and SOS / Detect Threat alert displays.

AI-Assisted Development Tools

During development, the team also used several AI-assisted development and ideation tools to improve productivity, accelerate prototyping, and refine architecture decisions. These included ChatGPT, GitHub Copilot, Google Gemini, Claude, and Lovable. Lovable was primarily used for dashboard interface inspiration, organizational workflow ideation, and UI concept exploration. AI-assisted tools were used to support development efficiency, documentation refinement, brainstorming, and interface exploration, while all architectural decisions, workflow designs, and implementation logic remained team-directed.

Free vs. Paid Tools

The project was built entirely using free-tier tools and student-accessible development resources.

Free tools used:

- TensorFlow
- Keras
- Flask
- MongoDB
- React Native (Expo)
- Railway (free tier hosting)
- GitHub Copilot (student/trial access where applicable)
- Gemini API (free tier)
- Claude (free tier access)
- Lovable (free usage)

Paid tools used: none. The team did not rely on any paid enterprise development tools during the hackathon.